

Primer Parcial Práctico

Cinemática Directa e Inversa del Robot Elite Robots CS66 con ROS 2 Jazzy

Grupo 2

Nathalia Villegas

Thais Mayta

IMT-342 Robótica – Universidad Católica Boliviana “San Pablo”

Docente: Carlos Daniel Aguilar Mancachi

Resumen—En este trabajo se desarrolla el modelo de cinemática directa e inversa del robot colaborativo Elite Robots CS66 utilizando ROS 2 Jazzy. El modelo cinemático se obtuvo mediante la convención Denavit–Hartenberg estándar, considerando los seis grados de libertad del manipulador. A partir de las transformaciones homogéneas se obtuvo la posición y orientación del efector final mediante la cinemática directa. Para la cinemática inversa de posición se implementó un método numérico iterativo basado en el Jacobiano posicional y su pseudoinversa. El Jacobiano se calculó numéricamente mediante diferencias finitas y la actualización articular se realizó mediante un factor de paso $\alpha = 0.5$. La implementación se integró en ROS 2 mediante los tópicos `/target` y `/joint_states`, permitiendo visualizar las soluciones en RViz2. Se realizaron tres validaciones de cinemática directa y tres pruebas de cinemática inversa. Los tres objetivos de cinemática inversa fueron alcanzados con errores inferiores a 10^{-4} m, obteniéndose errores finales de 0.000074 m, 0.000052 m y 0.000084 m.

Index Terms—Denavit–Hartenberg, cinemática directa, cinemática inversa, Jacobiano, ROS 2, RViz2, manipulador serial.

I. DESCRIPCIÓN DEL ROBOT ASIGNADO

El robot asignado al Grupo 2 es el manipulador serial Elite Robots CS66. Para el desarrollo de este trabajo se consideran sus seis grados de libertad, correspondientes a seis articulaciones de tipo revoluta. El modelo cinemático se implementa utilizando la convención Denavit–Hartenberg estándar y se emplean unidades del Sistema Internacional, es decir, metros para las distancias y radianes para las variables angulares.

I-A. Datos generales

Cuadro I
DATOS PRINCIPALES DEL ROBOT

Dato	Valor
Fabricante	Elite Robots
Modelo	CS66
Grados de libertad	6
Tipo de articulaciones	Revolutas
Unidades empleadas	SI: metros y radianes

I-B. Cadena cinemática

La cadena cinemática considerada está formada por seis articulaciones revolutas, cuyas variables articulares se representan mediante q_1, q_2, q_3, q_4, q_5 y q_6 . La secuencia de transformaciones empleada para relacionar la base con el efector final es

$$T_0^6 = A_0^1 A_1^2 A_2^3 A_3^4 A_4^5 A_5^6. \quad (1)$$



Figura 1. Robot Elite Robots CS66 y asignación de las articulaciones consideradas en el modelo cinemático.

II. ASIGNACIÓN DE MARCOS DE REFERENCIA

La asignación de marcos de referencia se realizó siguiendo la convención Denavit–Hartenberg estándar. Para cada articulación revoluta, el eje z_{i-1} se asocia con el eje de movimiento de la articulación correspondiente. Los ejes x_i se establecen de acuerdo con la normal común entre los ejes consecutivos, mientras que los ejes y_i se obtienen mediante la regla de la mano derecha.

La cadena utiliza seis marcos asociados a las seis articulaciones del manipulador. Los desplazamientos y giros relativos entre marcos se representan mediante los parámetros d_i , a_i y α_i , mientras que las variables articulares corresponden a los ángulos θ_i .

En particular, debido a la orientación seleccionada para el primer marco, la primera variable articular presenta un desplazamiento angular constante, por lo que:

$$\theta_1 = q_1 + \pi. \quad (2)$$

Para las restantes articulaciones se emplean:

$$\theta_i = q_i, \quad i = 2, \dots, 6. \quad (3)$$

La asignación utilizada se muestra en la Figura 2.

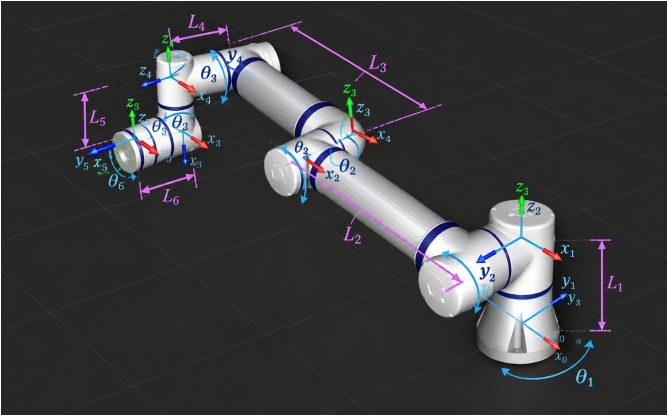


Figura 2. Asignación de marcos de referencia utilizada para el modelo cinemático del Elite Robots CS66.

Debido a la orientación seleccionada para el primer marco, la primera variable articular presenta un desplazamiento angular constante, por lo que:

$$\theta_1 = q_1 + \pi. \quad (4)$$

Para las restantes articulaciones se emplea:

$$\theta_i = q_i, \quad i = 2, \dots, 6. \quad (5)$$

II-A. Convención utilizada

En este trabajo se utiliza la convención Denavit–Hartenberg estándar. La transformación homogénea entre dos marcos consecutivos se expresa como

$$A_{i-1}^i = R_z(\theta_i) T_z(d_i) T_x(a_i) R_x(\alpha_i). \quad (6)$$

Su forma matricial es

$$A_{i-1}^i = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} c_{\alpha_i} & s_{\theta_i} s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i} c_{\alpha_i} & -c_{\theta_i} s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (7)$$

donde

$$c_{\theta_i} = \cos(\theta_i), \quad s_{\theta_i} = \sin(\theta_i), \quad (8)$$

y

$$c_{\alpha_i} = \cos(\alpha_i), \quad s_{\alpha_i} = \sin(\alpha_i). \quad (9)$$

III. PARÁMETROS DENAVIT–HARTENBERG

Los parámetros Denavit–Hartenberg se obtuvieron a partir de la geometría del manipulador y de la asignación de marcos mostrada anteriormente. La primera articulación presenta un desplazamiento angular constante de π debido a la orientación seleccionada para los marcos.

III-A. Verificación geométrica de los parámetros

Los parámetros geométricos empleados en el modelo corresponden a las distancias representadas en la asignación de marcos del manipulador. Las longitudes utilizadas son:

$$L_1 = 0.1625 \text{ m}, \quad (10)$$

$$L_2 = 0.4270 \text{ m}, \quad (11)$$

$$L_3 = 0.3905 \text{ m}, \quad (12)$$

$$L_4 = 0.1475 \text{ m}, \quad (13)$$

$$L_5 = 0.0965 \text{ m}, \quad (14)$$

$$L_6 = 0.0950 \text{ m}. \quad (15)$$

Estas dimensiones se incorporan en los parámetros d_i y a_i de la Tabla II. Los valores angulares α_i se determinan a partir de la orientación relativa de los ejes z de los marcos consecutivos.

La corrección angular de la primera articulación se incorpora directamente en la variable θ_1 , de acuerdo con:

$$\theta_1 = q_1 + \pi. \quad (16)$$

IV. CINEMÁTICA DIRECTA

La cinemática directa permite determinar la posición y orientación del efector final a partir de la configuración articular del manipulador.

Para un manipulador de seis grados de libertad, la transformación total desde la base hasta el efector final se obtiene mediante el producto de las transformaciones homogéneas individuales:

$$T_0^6 = A_0^1 A_1^2 A_2^3 A_3^4 A_4^5 A_5^6. \quad (17)$$

La transformación resultante puede expresarse como

$$T_0^6 = \begin{bmatrix} R_0^6 & p_0^6 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (18)$$

donde R_0^6 representa la orientación del efector final respecto al marco base y p_0^6 representa su posición.

Cuadro II
PARÁMETROS DENAVIT–HARTENBERG DEL ROBOT ELITE ROBOTS CS66

Transformación	θ_i	d_i [m]	a_i [m]	α_i [rad]	Descripción
A_0^1	$q_1 + \pi$	0.1625	0	$-\pi/2$	base $\rightarrow$ link 1
A_1^2	q_2	0	0.427	0	link 1 $\rightarrow$ link 2
A_2^3	q_3	0.1475	0.3905	0	link 2 $\rightarrow$ link 3
A_3^4	q_4	0	0	$-\pi/2$	link 3 $\rightarrow$ link 4
A_4^5	q_5	0.0965	0	$\pi/2$	link 4 $\rightarrow$ link 5
A_5^6	q_6	0.095	0	0	link 5 $\rightarrow$ efector

IV-A. Matrices homogéneas individuales

A partir de los parámetros de la Tabla II, las matrices de transformación individuales quedan definidas de la siguiente manera.

$$A_0^1 = \begin{bmatrix} -\cos q_1 & 0 & \sin q_1 & 0 \\ -\sin q_1 & 0 & -\cos q_1 & 0 \\ 0 & -1 & 0 & 0.1625 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (19)$$

$$A_1^2 = \begin{bmatrix} -\cos q_1 & 0 & \sin q_1 & 0 \\ -\sin q_1 & 0 & -\cos q_1 & 0 \\ 0 & -1 & 0 & 0.1625 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (20)$$

$$A_2^3 = \begin{bmatrix} \cos q_2 & -\sin q_2 & 0 & 0.427 \cos q_2 \\ \sin q_2 & \cos q_2 & 0 & 0.427 \sin q_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (21)$$

$$A_3^4 = \begin{bmatrix} \cos q_3 & -\sin q_3 & 0 & 0.3905 \cos q_3 \\ \sin q_3 & \cos q_3 & 0 & 0.3905 \sin q_3 \\ 0 & 0 & 1 & 0.1475 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (22)$$

$$A_4^5 = \begin{bmatrix} \cos q_4 & 0 & -\sin q_4 & 0 \\ \sin q_4 & 0 & \cos q_4 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (23)$$

$$A_5^6 = \begin{bmatrix} \cos q_5 & 0 & \sin q_5 & 0 \\ \sin q_5 & 0 & -\cos q_5 & 0 \\ 0 & 1 & 0 & 0.0965 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (24)$$

$$A_6^7 = \begin{bmatrix} \cos q_6 & -\sin q_6 & 0 & 0 \\ \sin q_6 & \cos q_6 & 0 & 0 \\ 0 & 0 & 1 & 0.095 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (25)$$

IV-B. Matriz de transformación total

Para simplificar la expresión de la transformación total se definen las siguientes variables auxiliares:

$$c_i = \cos(q_i), \quad s_i = \sin(q_i), \quad (26)$$

$$c_{23} = \cos(q_2 + q_3), \quad s_{23} = \sin(q_2 + q_3), \quad (27)$$

$$c_{234} = \cos(q_2 + q_3 + q_4), \quad s_{234} = \sin(q_2 + q_3 + q_4). \quad (28)$$

El producto de las seis matrices homogéneas produce:

$$T_0^6 = \begin{bmatrix} r_{11} & r_{12} & r_{13} & p_x \\ r_{21} & r_{22} & r_{23} & p_y \\ r_{31} & r_{32} & r_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (29)$$

$$p_x = c_1 (-0.427c_2 - 0.3905c_{23} + 0.0965s_{234} - 0.095s_5c_{234}) + s_1 (0.1475 + 0.095c_5), \quad (30)$$

$$p_y = s_1 (-0.427c_2 - 0.3905c_{23} + 0.0965s_{234} - 0.095s_5c_{234}) - c_1 (0.1475 + 0.095c_5), \quad (31)$$

$$p_z = 0.1625 - 0.427s_2 - 0.3905s_{23} - 0.0965c_{234} - 0.095s_5s_{234}. \quad (32)$$

IV-C. Posición y orientación del efector

La posición del efector final se obtiene de la cuarta columna de la matriz de transformación total. Por tanto,

$$p(q) = \begin{bmatrix} p_x(q) \\ p_y(q) \\ p_z(q) \end{bmatrix}, \quad (33)$$

donde las componentes están dadas por

$$p_x(q) = c_1 (-0.427c_2 - 0.3905c_{23} + 0.0965s_{234} - 0.095s_5c_{234}) + s_1 (0.1475 + 0.095c_5), \quad (34)$$

$$p_y(q) = s_1 (-0.427c_2 - 0.3905c_{23} + 0.0965s_{234} - 0.095s_5c_{234}) - c_1 (0.1475 + 0.095c_5), \quad (35)$$

$$p_z(q) = 0.1625 - 0.427s_2 - 0.3905s_{23} - 0.0965c_{234} - 0.095s_5s_{234}. \quad (36)$$

La orientación del efector final está representada por la matriz de rotación R_0^6 :

$$R_0^6 = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}, \quad (37)$$

donde, utilizando las variables auxiliares definidas anteriormente,

$$r_{11} = -(s_1 s_5 + c_1 c_5 c_{234}) c_6 + c_1 s_{234} s_6, \quad (38)$$

$$r_{12} = (s_1 s_5 + c_1 c_5 c_{234}) s_6 + c_1 s_{234} c_6, \quad (39)$$

$$r_{13} = s_1 c_5 - c_1 s_5 c_{234}, \quad (40)$$

$$r_{21} = -(s_1 c_5 c_{234} - c_1 s_5) c_6 + s_1 s_{234} s_6, \quad (41)$$

$$r_{22} = (s_1 c_5 c_{234} - c_1 s_5) s_6 + s_1 s_{234} c_6, \quad (42)$$

$$r_{23} = -s_1 s_5 c_{234} - c_1 c_5, \quad (43)$$

$$r_{31} = -s_{234} s_6 - c_5 c_{234} c_6, \quad (44)$$

$$r_{32} = c_5 s_{234} s_6 - c_{234} c_6, \quad (45)$$

$$r_{33} = -s_5 s_{234}. \quad (46)$$

IV-D. Comprobaciones matemáticas

La matriz de rotación obtenida debe cumplir las propiedades de una matriz de rotación. Por ello se verificará que:

$$(R_0^6)^T R_0^6 \approx I, \quad (47)$$

y

$$\det(R_0^6) \approx 1. \quad (48)$$

Asimismo, toda matriz de transformación homogénea utilizada debe presentar la última fila en la forma

$$[0 \ 0 \ 0 \ 1]. \quad (49)$$

La verificación numérica se realizará posteriormente para una configuración articular específica del robot.

V. JACOBIANO DEL MANIPULADOR

Para la cinemática inversa de posición se emplea el Jacobiano posicional, que relaciona pequeñas variaciones de las variables articulares con variaciones de la posición del efector final.

El Jacobiano posicional se define como

$$J_v(q) = \begin{bmatrix} \frac{\partial p_x}{\partial q_1} & \frac{\partial p_x}{\partial q_2} & \frac{\partial p_x}{\partial q_3} & \frac{\partial p_x}{\partial q_4} & \frac{\partial p_x}{\partial q_5} & \frac{\partial p_x}{\partial q_6} \\ \frac{\partial p_y}{\partial q_1} & \frac{\partial p_y}{\partial q_2} & \frac{\partial p_y}{\partial q_3} & \frac{\partial p_y}{\partial q_4} & \frac{\partial p_y}{\partial q_5} & \frac{\partial p_y}{\partial q_6} \\ \frac{\partial p_z}{\partial q_1} & \frac{\partial p_z}{\partial q_2} & \frac{\partial p_z}{\partial q_3} & \frac{\partial p_z}{\partial q_4} & \frac{\partial p_z}{\partial q_5} & \frac{\partial p_z}{\partial q_6} \end{bmatrix}. \quad (50)$$

V-A. Jacobiano utilizado en el proyecto

En la implementación desarrollada, el Jacobiano posicional se obtiene numéricamente mediante diferencias finitas. Para cada articulación q_i se introduce una pequeña perturbación ε y se calcula nuevamente la posición del efector final.

La aproximación utilizada es:

$$\frac{\partial p}{\partial q_i} \approx \frac{p(q + \varepsilon e_i) - p(q)}{\varepsilon}, \quad (51)$$

donde e_i es el vector unitario asociado a la articulación i y $\varepsilon = 10^{-6}$.

Por tanto, la columna i del Jacobiano se calcula como:

$$J_{v,i} = \frac{p(q + \varepsilon e_i) - p(q)}{\varepsilon}. \quad (52)$$

De esta manera se construye una matriz de dimensiones 3×6 :

$$J_v(q) = \begin{bmatrix} \frac{\partial p_x}{\partial q_1} & \dots & \frac{\partial p_x}{\partial q_6} \\ \frac{\partial p_y}{\partial q_1} & \dots & \frac{\partial p_y}{\partial q_6} \\ \frac{\partial p_z}{\partial q_1} & \dots & \frac{\partial p_z}{\partial q_6} \end{bmatrix}. \quad (53)$$

El cálculo numérico se implementa mediante diferencias finitas en el nodo de cinemática inversa, utilizando una perturbación $\varepsilon = 10^{-6}$.

$$J_v(q) = \begin{bmatrix} J_{11} & J_{12} & J_{13} & J_{14} & J_{15} & J_{16} \\ J_{21} & J_{22} & J_{23} & J_{24} & J_{25} & J_{26} \\ J_{31} & J_{32} & J_{33} & J_{34} & J_{35} & J_{36} \end{bmatrix}$$

donde:

$$J_{11} = s_1(0.427c_2 + 0.3905c_{23} - 0.0965s_{234} + 0.095s_5c_{234}) + c_1(0.1475 + 0.095c_5)$$

$$J_{12} = c_1(0.427s_2 + 0.3905s_{23} + 0.0965c_{234} + 0.095s_5s_{234})$$

$$J_{13} = c_1(0.3905s_{23} + 0.0965c_{234} + 0.095s_5s_{234})$$

$$J_{14} = c_1(0.0965c_{234} + 0.095s_5s_{234})$$

$$J_{15} = -c_1(0.095s_5c_{234}) - s_1(0.095s_5)$$

$$J_{16} = 0$$

$$J_{21} = c_1(0.427c_2 + 0.3905c_{23} - 0.0965s_{234} + 0.095s_5c_{234}) - s_1(0.1475 + 0.095c_5)$$

$$J_{22} = s_1(0.427s_2 + 0.3905s_{23} + 0.0965c_{234} + 0.095s_5s_{234})$$

$$J_{23} = s_1(0.3905s_{23} + 0.0965c_{234} + 0.095s_5s_{234})$$

$$J_{24} = s_1(0.0965c_{234} + 0.095s_5s_{234})$$

$$J_{25} = -s_1(0.095c_5c_{234}) + c_1(0.095s_5)$$

$$J_{26} = 0$$

$$J_{31} = 0$$

$$J_{32} = -0.427c_2 - 0.3905c_{23} + 0.0965s_{234} - 0.095s_5c_{234}$$

$$J_{33} = -0.3905c_{23} + 0.0965s_{234} - 0.095s_5c_{234}$$

$$J_{34} = 0.0965s_{234} - 0.095s_5c_{234}$$

$$J_{35} = -0.095c_5s_{234}$$

$$J_{36} = 0$$

donde:

$$c_{23} = \cos(q_2 + q_3), \quad s_{23} = \sin(q_2 + q_3)$$

$$c_{234} = \cos(q_2 + q_3 + q_4), \quad s_{234} = \sin(q_2 + q_3 + q_4)$$

V-B. Consideración de singularidades

Una configuración singular ocurre cuando el Jacobiano pierde rango, reduciendo el número de direcciones cartesianas que pueden ser producidas por pequeñas variaciones de las variables articulares.

En el método iterativo utilizado, una configuración cercana a una singularidad puede producir una pseudoinversa con valores numéricamente grandes y, como consecuencia, generar incrementos articulares elevados o una convergencia más lenta. Por este motivo, el comportamiento del método depende de la configuración inicial y de la proximidad del robot a configuraciones singulares.

En esta implementación se utiliza la pseudoinversa numérica calculada mediante `numpy.linalg.pinv`, sin incorporar un método de regularización adicional.

VI. CINEMÁTICA INVERSA

VI-A. Definición del problema

La cinemática inversa tiene como objetivo encontrar una configuración articular que permita alcanzar una posición cartesiana deseada del efector final.

El objetivo cartesiano se define como:

$$p_d = \begin{bmatrix} x_d \\ y_d \\ z_d \end{bmatrix}. \quad (54)$$

A partir de esta posición, el algoritmo busca una configuración articular

$$q^* = [q_1 \ q_2 \ q_3 \ q_4 \ q_5 \ q_6]^T \quad (55)$$

que satisfaga aproximadamente:

$$p(q^*) \approx p_d. \quad (56)$$

En este trabajo la cinemática inversa considera únicamente la posición cartesiana del efector final. La orientación se obtiene mediante la cinemática directa y no forma parte del objetivo de la IK.

VI-B. Objetivo recibido mediante ROS 2

El objetivo cartesiano se recibe mediante el tópic

`/target`

utilizando mensajes del tipo

`geometry_msgs/msg/Point`.

El mensaje contiene directamente las coordenadas cartesianas deseadas:

$$p_d = \begin{bmatrix} \text{msg.x} \\ \text{msg.y} \\ \text{msg.z} \end{bmatrix}. \quad (57)$$

De esta manera, el nodo de cinemática inversa puede recibir nuevos objetivos sin modificar directamente el código.

VI-C. Método numérico iterativo

Para resolver la cinemática inversa se utiliza un método numérico iterativo basado en el Jacobiano posicional y su pseudoinversa.

En cada iteración k se calcula el error de posición como

$$e_k = p_d - p(q_k). \quad (58)$$

El Jacobiano se evalúa en la configuración actual:

$$J_k = J_v(q_k). \quad (59)$$

La actualización de las variables articulares se realiza mediante:

$$q_{k+1} = q_k + \alpha J_k^+ e_k, \quad (60)$$

donde α corresponde al factor de actualización y J_k^+ es la pseudoinversa del Jacobiano.

En la implementación se utiliza

$$\alpha = 0.5. \quad (61)$$

La pseudoinversa se calcula numéricamente mediante la función `numpy.linalg.pinv` de NumPy.

VI-D. Configuración inicial

El algoritmo inicia a partir de una configuración articular inicial definida en el nodo de cinemática inversa. La configuración utilizada es:

$$q_0 = \begin{bmatrix} 0 & -\frac{\pi}{4} & \frac{\pi}{2} & -\frac{\pi}{4} & 0 & 0 \end{bmatrix}^T \text{ rad.} \quad (62)$$

Esta configuración se utiliza como semilla inicial para el proceso iterativo. La validez de la configuración respecto a los límites articulares del modelo será verificada mediante los límites definidos en el URDF/XACRO.

VI-E. Articulaciones utilizadas

La solución numérica utiliza las seis articulaciones del manipulador:

$$q = [q_1 \ q_2 \ q_3 \ q_4 \ q_5 \ q_6]^T. \quad (63)$$

Ninguna de las seis articulaciones se mantiene fija durante el proceso de cinemática inversa. El Jacobiano utilizado posee, por tanto, tres filas correspondientes a la posición cartesiana y seis columnas correspondientes a las seis variables articulares.

VI-F. Límites articulares

Para el proceso de cinemática inversa se consideran los límites articulares de las seis articulaciones del robot Elite Robots CS66. De acuerdo con la documentación oficial del fabricante, cada una de las seis articulaciones presenta un rango de movimiento de $\pm 360^\circ$, equivalente a $\pm 2\pi$ rad.

Por tanto, los límites articulares utilizados como referencia en el modelo son:

Cuadro III
LÍMITES ARTICULARES UTILIZADOS

Joint	Mínimo [rad]	Máximo [rad]
q_1	-2π	2π
q_2	-2π	2π
q_3	-2π	2π
q_4	-2π	2π
q_5	-2π	2π
q_6	-2π	2π

Las configuraciones articulares obtenidas durante las pruebas de cinemática inversa se encuentran dentro de estos límites. Estos valores se consideran como referencia para evaluar la validez de las soluciones obtenidas por el método iterativo.

VI-G. Criterio de convergencia

El error de posición se calcula mediante la norma euclidiana:

$$e_p = \|p_d - p(q_k)\|_2. \quad (64)$$

El algoritmo considera que se alcanzó la convergencia cuando:

$$e_p < \varepsilon. \quad (65)$$

En la implementación se utilizaron los siguientes parámetros:

$$\varepsilon = 10^{-4} \text{ m}, \quad (66)$$

$$\alpha = 0.5, \quad (67)$$

$$N_{\text{máx}} = 200. \quad (68)$$

El proceso iterativo finaliza cuando el error es menor que la tolerancia o cuando se alcanza el número máximo de iteraciones.

VI-H. Publicación de la solución articular

Una vez obtenida la solución de la cinemática inversa, las posiciones articulares calculadas se publican mediante el tópic:

/joint_states

utilizando mensajes:

sensor_msgs/msg/JointState.

El mensaje contiene los nombres y posiciones de las seis articulaciones:

```
shoulder_pan_joint
shoulder_lift_joint
elbow_joint
wrist_1_joint
wrist_2_joint
wrist_3_joint
```

La publicación periódica de estos valores permite que robot_state_publisher actualice la configuración del robot y que la solución obtenida sea visualizada en RViz2.

VI-I. Verificación mediante cinemática directa

La configuración articular obtenida mediante la cinemática inversa se verifica nuevamente mediante la cinemática directa.

Para la solución encontrada q^* , se calcula:

$$p_f = p(q^*). \quad (69)$$

Posteriormente se obtiene el error final:

$$e_f = \|p_d - p_f\|_2. \quad (70)$$

La solución se considera correctamente alcanzada cuando:

$$e_f < 10^{-4} \text{ m}. \quad (71)$$

El nodo de cinemática directa permite calcular nuevamente la posición del efector final a partir de la configuración articular publicada por el nodo de IK.

VII. IMPLEMENTACIÓN EN PYTHON Y ROS 2

VII-A. Herramientas

La implementación del proyecto se realizó utilizando Ubuntu 24.04 y ROS 2 Jazzy como entorno principal de desarrollo. Para la visualización y validación del manipulador se utilizó RViz2 junto con los nodos robot_state_publisher y joint_state_publisher_gui.

Los cálculos matriciales y la implementación de las funciones de cinemática se realizaron en Python utilizando NumPy. El modelo cinemático empleado utiliza los parámetros Denavit–Hartenberg obtenidos para el robot Elite Robots CS66.

VII-B. Nodo de cinemática directa

Se desarrolló un nodo propio denominado `fk_node.py`, encargado de calcular la cinemática directa del manipulador a partir de los parámetros Denavit–Hartenberg definidos previamente.

El nodo se suscribe al tópic `/joint_states`, desde donde recibe las posiciones articulares. Las articulaciones se ordenan de acuerdo con los nombres definidos en el modelo del robot:

- `shoulder_pan_joint`;
- `shoulder_lift_joint`;
- `elbow_joint`;
- `wrist_1_joint`;
- `wrist_2_joint`;
- `wrist_3_joint`.

Para cada articulación se construye la matriz homogénea correspondiente mediante la transformación DH. Posteriormente, las seis transformaciones se multiplican para obtener la transformación total:

$$T_0^6 = A_0^1 A_1^2 A_2^3 A_3^4 A_4^5 A_5^6. \quad (72)$$

A partir de la transformación total se extrae el vector de posición del efector final:

$$p = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = T_0^6[0 : 3, 3]. \quad (73)$$

El resultado se muestra en la terminal indicando los valores articulares y las coordenadas cartesianas calculadas.

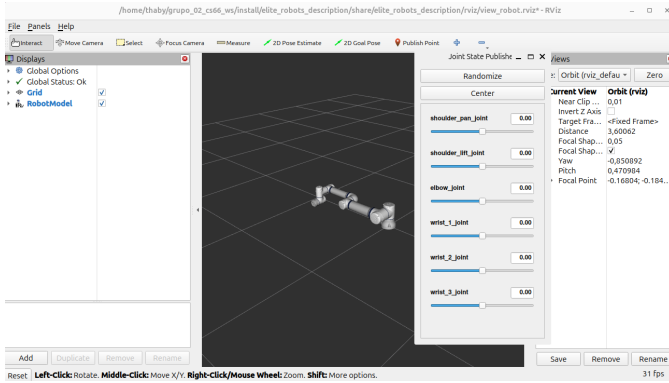


Figura 3. Visualización del robot Elite Robots CS66 en RViz2 mediante el modelo del robot y Joint State Publisher GUI.

VII-C. Nodo de cinemática inversa

Se desarrolló el nodo propio `ik_node.py` para resolver la cinemática inversa de posición del robot. El nodo recibe un objetivo cartesiano mediante el tópic `/target`, utilizando mensajes del tipo `geometry_msgs/msg/Point`.

El objetivo recibido se expresa como:

$$p_d = \begin{bmatrix} x_d \\ y_d \\ z_d \end{bmatrix}. \quad (74)$$

A partir de una configuración articular inicial, el nodo calcula la posición actual mediante la cinemática directa y obtiene el error cartesiano:

$$e_k = p_d - p(q_k). \quad (75)$$

Posteriormente se calcula el Jacobiano posicional mediante diferencias finitas y se obtiene su pseudoinversa numérica utilizando `numpy.linalg.pinv`. La actualización de las articulaciones se realiza mediante:

$$q_{k+1} = q_k + \alpha J_v^+(q_k) e_k. \quad (76)$$

El proceso se repite hasta que el error de posición es menor que la tolerancia establecida o hasta alcanzar el número máximo de iteraciones.

Una vez obtenida la solución, esta se almacena como configuración actual y se publica mediante el tópic `/joint_states`, permitiendo que `robot_state_publisher` actualice el modelo visualizado en RViz2.

VII-D. Interacción con Joint State Publisher GUI

El nodo `joint_state_publisher_gui` se utilizó para modificar manualmente las posiciones articulares del robot durante las pruebas de cinemática directa. Los valores seleccionados mediante los controles del GUI son publicados en `/joint_states`, permitiendo que el nodo `fk_node.py` reciba las configuraciones articulares y calcule la posición del efector final.

Para las pruebas de cinemática inversa se utilizó el nodo `ik_node.py` como publicador de `/joint_states`, evitando utilizar simultáneamente el GUI como segundo publicador de estados articulares.

VIII. WORKSPACE ROS 2

VIII-A. Estructura del workspace

El proyecto fue desarrollado dentro del workspace:

`~/grupo_02_cs66_ws`

La estructura principal utilizada durante el desarrollo fue:

```
grupo_02_cs66_ws/
|-- src/
|-- build/
|-- install/
|-- log/
'-- entorno.sh
```

Dentro de la carpeta `src/` se encuentran los paquetes utilizados para la descripción, visualización y cálculo cinemático del robot.

Los directorios `build/`, `install/` y `log/` corresponden a archivos generados durante la compilación y ejecución del workspace.

VIII-B. Compilación y ejecución

La compilación del workspace se realizó mediante:

```
cd ~/grupo_02_cs66_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
```

Posteriormente se cargó el entorno específico del proyecto:

```
source entorno.sh
```

La visualización del robot se ejecutó mediante:

```
ros2 launch grupo02_cs66_bringup display.launch.py
```

El nodo de cinemática directa se ejecutó mediante:

```
ros2 run grupo02_cs66_kinematics fk_node
```

Mientras que el nodo de cinemática inversa se ejecutó mediante:

```
ros2 run grupo02_cs66_kinematics ik_node
```

IX. VALIDACIÓN DE LA CINEMÁTICA DIRECTA

Se realizaron tres pruebas con diferentes configuraciones articulares utilizando el Joint State Publisher GUI. Para cada configuración, el nodo `fk_node.py` recibió las posiciones articulares mediante `/joint_states` y calculó la posición del efector final mediante las matrices DH.

Cuadro IV
RESULTADOS DE LA VALIDACIÓN DE LA CINEMÁTICA DIRECTA.

Caso	q [rad]	$PROS$ [m]
1	[0, 0, 0, 0, 0, 0]	[-0.8175, -0.2425, 0.0660]
2	[0.5, 0.5, 0.5, 0, 0, 0]	[-0.3260, -0.4547, -0.4232]
3	[-0.5, 0.301, -0.4, 0.5, 0.301, -0.5]	[-0.8028, 0.1675, -0.0248]

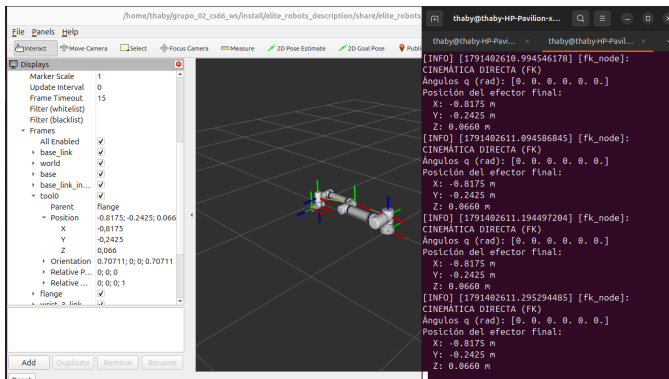


Figura 4. Validación de la cinemática directa para la configuración 1.

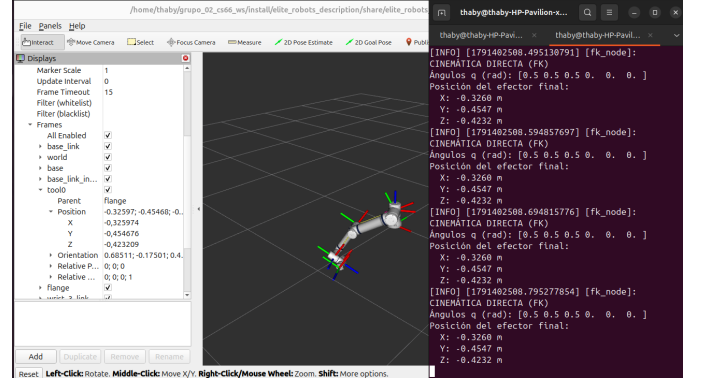


Figura 5. Validación de la cinemática directa para la configuración 2.

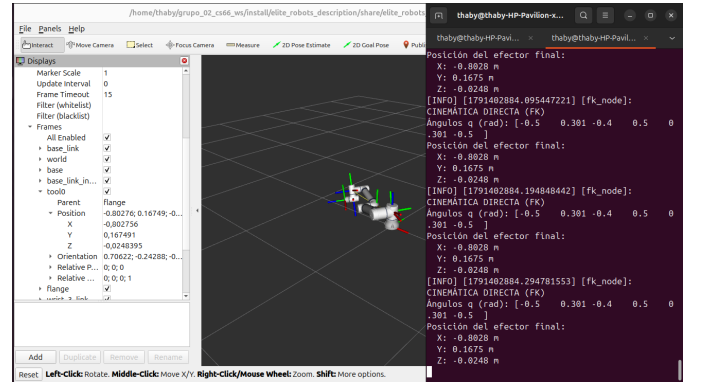


Figura 6. Validación de la cinemática directa para la configuración 3.

X. VALIDACIÓN DE LA CINEMÁTICA INVERSA

Se realizaron tres pruebas de cinemática inversa utilizando objetivos cartesianos alcanzables obtenidos previamente mediante las pruebas de cinemática directa. Cada objetivo fue enviado mediante el tópico `/target` utilizando mensajes `geometry_msgs/msg/Point`.

El criterio de convergencia utilizado por el algoritmo fue:

$$\|p_d - p(q_k)\|_2 < 10^{-4} \text{ m.} \quad (77)$$

Los tres casos evaluados alcanzaron el objetivo con un error inferior a la tolerancia establecida.

Cuadro V
RESULTADOS DE LAS PRUEBAS DE CINEMÁTICA INVERSA.

Caso	p_d [m]	q^* [rad]
1	[-0.8175, -0.2425, 0.0660]	[0.004, -0.363, 0.843, -1.046, 0.265, 0]
2	[-0.3260, -0.4547, -0.4232]	[0.506, 0.238, 1.064, -1.048, 0.257, 0]
3	[-0.8028, 0.1675, -0.0248]	[-0.502, -0.290, 0.918, -1.040, 0.239, 0]

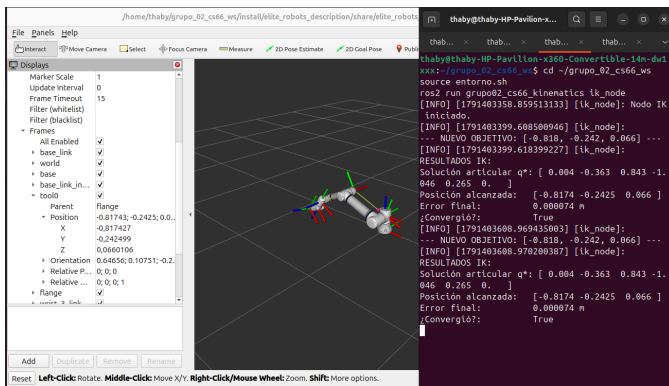


Figura 7. Resultado de la prueba de cinemática inversa para el objetivo 1.

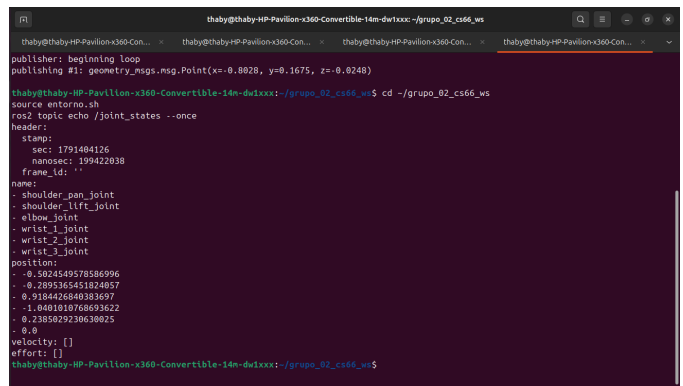


Figura 10. Publicación de la solución articular mediante el tópic `/joint_states`.

XI. REPOSITORIO GITHUB Y REPRODUCIBILIDAD

El código fuente desarrollado para el presente trabajo se encuentra disponible en un repositorio público de GitHub. El repositorio contiene los archivos necesarios para la implementación de la cinemática directa e inversa del robot Elite Robots CS66 utilizando ROS 2 Jazzy.

XI-A. Enlace

Repositorio: <https://github.com/NathaliaVillegas/Primer-Parcial-CS66>

Rama de entrega: `main`

El repositorio contiene los archivos fuente utilizados durante el desarrollo y validación del proyecto, incluyendo los nodos de cinemática, archivos de configuración y los elementos necesarios para la ejecución del proyecto en ROS 2.

XI-B. Contenido del repositorio

El repositorio incluye los archivos correspondientes a la implementación desarrollada por el grupo, así como los archivos necesarios para compilar y ejecutar los paquetes del proyecto. Entre los elementos principales se encuentran:

- código fuente de los nodos de cinemática directa e inversa;
- archivos de configuración del proyecto;
- archivos necesarios para la compilación de los paquetes ROS 2;
- archivos de lanzamiento utilizados para la visualización;
- archivos de configuración del entorno;
- documentación necesaria para reproducir la implementación.

XI-C. Archivos que no deben versionarse

Los directorios y archivos generados automáticamente durante la compilación o ejecución no forman parte del código fuente del proyecto. Entre ellos se encuentran:

```
build/
install/
log/
__pycache__/
```

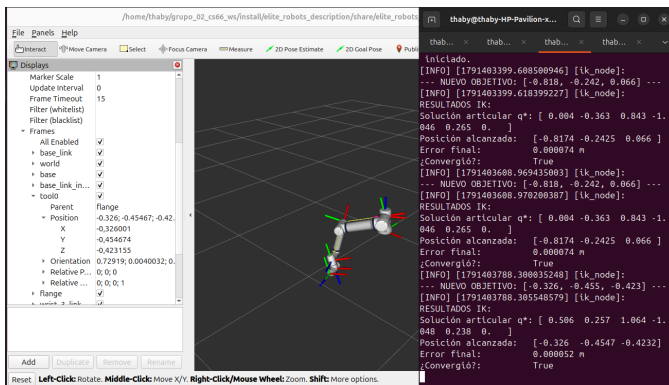


Figura 8. Resultado de la prueba de cinemática inversa para el objetivo 2.

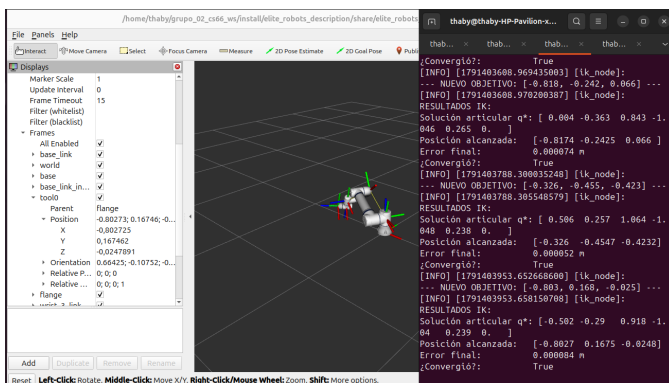


Figura 9. Resultado de la prueba de cinemática inversa para el objetivo 3.

```
*.pyc
.vscode/
.idea/
```

Estos archivos pueden ser generados nuevamente al compilar el workspace y, por lo tanto, no son necesarios para reproducir la implementación.

XI-D. Reproducción del proyecto

Para utilizar el proyecto, primero se debe clonar el repositorio y acceder al directorio correspondiente:

```
git clone https://github.com/NathaliaVillegas/Primer-Parcial-CS66.git
cd Primer-Parcial-CS66
```

Posteriormente, se debe preparar el entorno de ROS 2 Jazzy y compilar el workspace mediante:

```
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
```

Una vez compilado el proyecto, se pueden ejecutar los nodos de cinemática directa e inversa de acuerdo con las instrucciones incluidas en el repositorio.

XI-E. README

El repositorio incluye un archivo README.md con las instrucciones necesarias para la instalación, compilación y ejecución del proyecto. En este archivo se especifican las dependencias, los comandos utilizados y el procedimiento para ejecutar la visualización del robot y los nodos de cinemática.

XII. RESULTADOS Y DISCUSIÓN

Los resultados obtenidos muestran que la implementación desarrollada permite relacionar las configuraciones articulares del robot con la posición cartesiana del efector final mediante la cinemática directa.

En las tres configuraciones utilizadas para la validación de FK se obtuvieron diferentes posiciones del efector final, verificando que el modelo implementado responde de manera consistente ante cambios en las variables articulares.

Para la cinemática inversa se utilizaron tres objetivos cartesianos obtenidos previamente mediante FK. En los tres casos el algoritmo alcanzó el objetivo con errores inferiores a 10^{-4} m. Los errores finales obtenidos fueron 0.000074 m, 0.000052 m y 0.000084 m para los casos 1, 2 y 3, respectivamente.

Estos resultados muestran una correspondencia adecuada entre el modelo cinemático implementado y la visualización del robot en RViz2. La utilización de los marcos de referencia mediante TF permitió además comprobar visualmente la configuración espacial del manipulador.

La solución de cinemática inversa depende de la configuración inicial utilizada por el algoritmo, debido a que el método empleado es iterativo y utiliza la pseudoinversa del Jacobiano. Por esta razón, diferentes configuraciones iniciales pueden conducir a diferentes soluciones articulares para un mismo objetivo cartesiano.

Como limitación de la implementación actual, el nodo de IK utilizado en las pruebas no registra explícitamente en consola el número de iteraciones realizadas ni aplica explícitamente una restricción de límites articulares durante la actualización. Por ello, estos aspectos deben considerarse como puntos de mejora de la implementación.

XIII. CONCLUSIONES

1. Se desarrolló el modelo cinemático del robot Elite Robots CS66 utilizando la convención Denavit–Hartenberg, permitiendo relacionar las variables articulares con la posición del efector final.
2. Se implementó un nodo propio de cinemática directa en ROS 2 que recibe las posiciones articulares mediante `/joint_states`, calcula las transformaciones homogéneas y obtiene la posición cartesiana del efector final.
3. Se implementó un método numérico iterativo de cinemática inversa basado en el Jacobiano posicional y su pseudoinversa. Las tres pruebas realizadas alcanzaron los objetivos cartesianos con errores inferiores a 10^{-4} m.
4. La integración con ROS 2 permitió validar el modelo mediante RViz2, TF y la publicación de las soluciones articulares en `/joint_states`, comprobando la correspondencia entre el modelo matemático y la representación del robot.
5. Como trabajo futuro, se puede mejorar el nodo de cinemática inversa incorporando explícitamente límites articulares, registro del número de iteraciones y métodos de regularización para mejorar el comportamiento del algoritmo cerca de configuraciones singulares.

REFERENCIAS

- [1] Elite Robots, “CS66 COBOT – Technical Documents,” Elite Robots. [Online]. Available: <https://www.eliterobots.com/cobots/cs66>. [Accessed: Oct. 7, 2026].
- [2] Elite Robots, “User Manual CS66 Ver. 2.13.2,” Elite Robots. [Online]. Available: <https://www.eliterobots.com/download>. [Accessed: Oct. 7, 2026].
- [3] Open Robotics, “ROS 2 Documentation – Jazzy Jalisco,” ROS 2 Documentation. [Online]. Available: <https://docs.ros.org/en/jazzy/>. [Accessed: Oct. 7, 2026].
- [4] Open Robotics, “robot_state_publisher,” ROS 2 Documentation. [Online]. Available: https://docs.ros.org/en/jazzy/p/robot_state_publisher/. [Accessed: Oct. 7, 2026].
- [5] NumPy Developers, “numpy.linalg.pinv,” NumPy Documentation. [Online]. Available: <https://numpy.org/doc/stable/reference/generated/numpy.linalg.pinv.html>. [Accessed: Oct. 7, 2026].